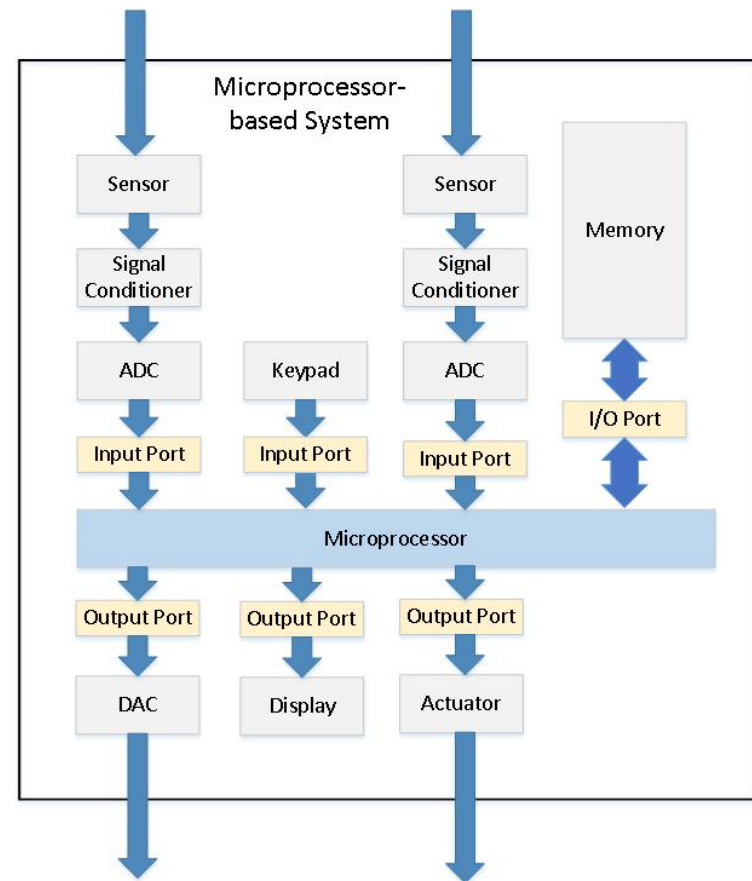


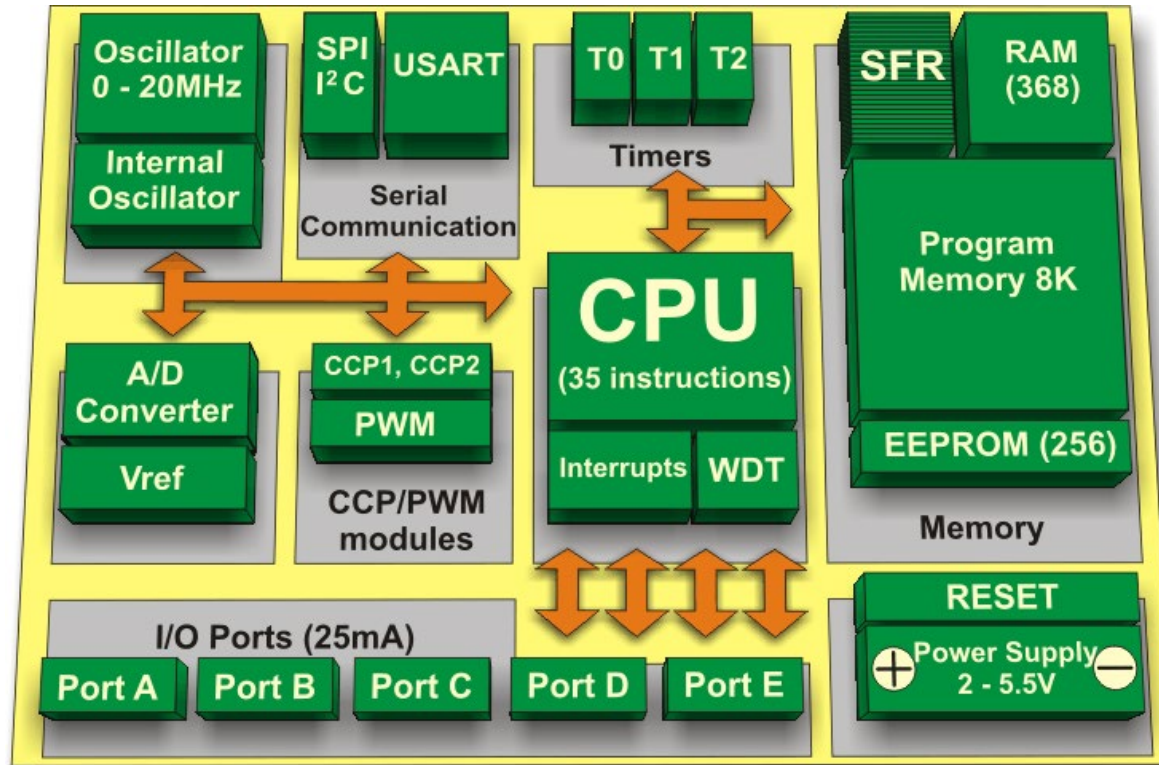
Learning Outcomes

- Understand how to connect any selected memory or peripheral devices to a microprocessor (e.g. with due regard to signal and timing requirements)
- Know how signals associated with sensors are processed via analogue circuits and acquired as quantized digital numbers using analogue-to-digital converters (ADC)
- Be able to write basic real-time programs to manipulate binary inputs from switches and ADC readings, and to output signals to implement user defined control operations

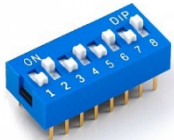


Microprocessor-based System

Microcontroller



Pushbutton
switch



Switches



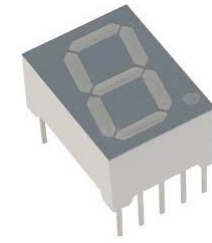
Keypad



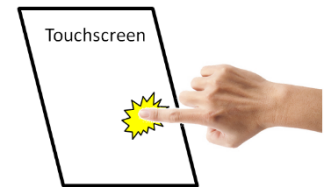
LEDs



Dot Matrix LED



7 segment display

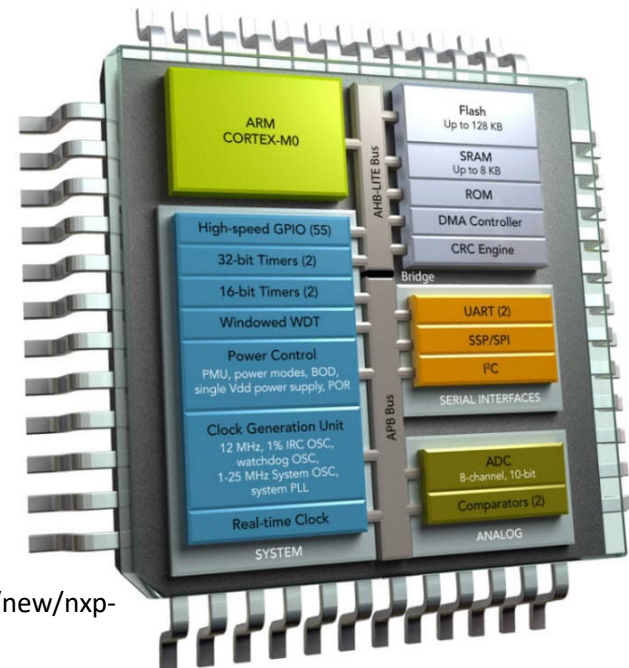


LCD



Timers and Counters

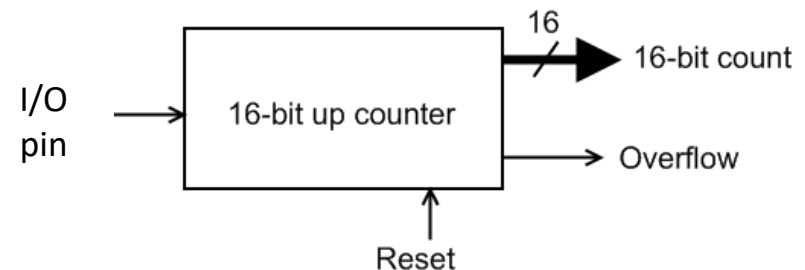
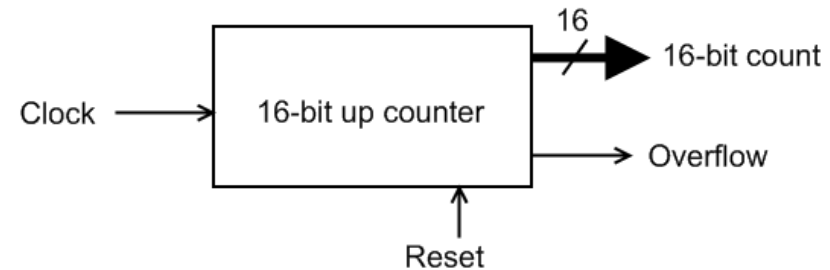
- Most embedded applications need to perform a variety of precision timer functions including:
 - Generate events at specific times
 - Determining the duration between two events
 - Count events.
- This can be achieved with the help of hardware timers/counters.
- The hardware timer/counter runs independently. It can count inputs or clock pulses concurrently while the microcontroller is executing the main program.



Source: <http://www.mouser.com/new/nxp-semiconductors/nxplpc1200/>

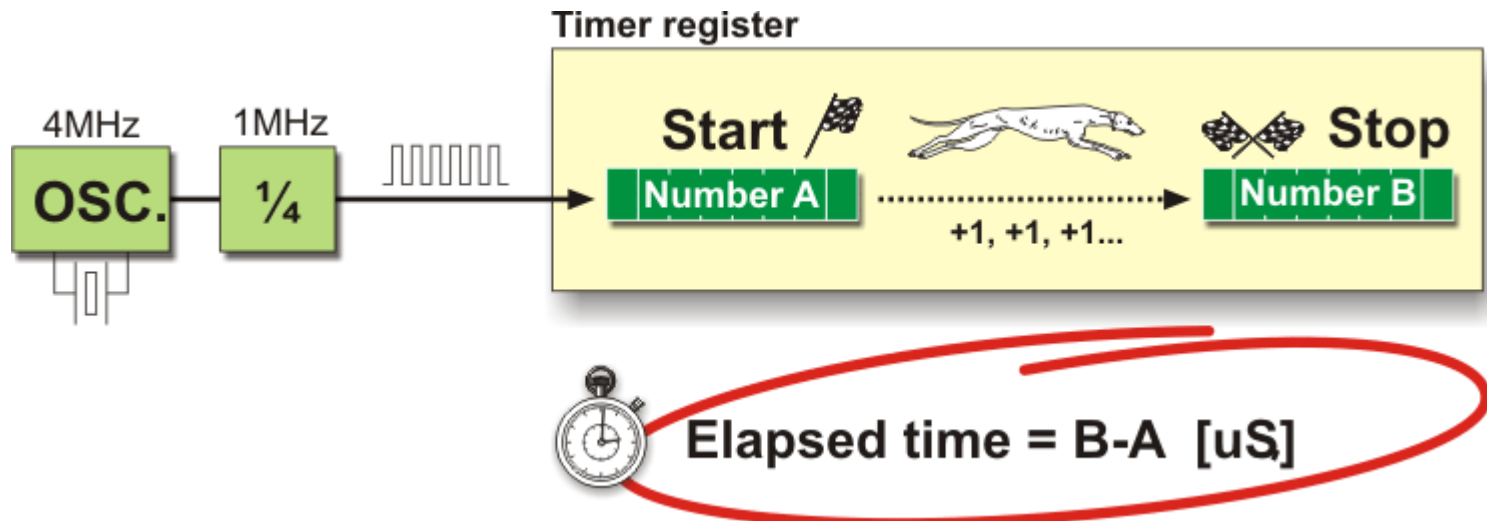
Timers vs. Counters

- Main component of a hardware timer/counter is a free running binary counter that increments at each incoming timing pulse.
- Timer:
 - Incoming pulse is a clock source with fixed frequency from the crystal oscillator.
 - Allows us to make accurate measurement of time interval by counting the pulses from the clock.
- Counter:
 - Incoming pulse is provided through external I/O pin of microcontroller.
 - Allows us to count external pulses which may appear at random intervals.
 - These pulses could be from a push button, sensors, etc.



How Does the Timer Work?

- Pulses are used to increment the number stored in the timer register.
- If the Timer register consists of 8 bits, the largest number it can store is 255. Based on the current oscillator setting below we can get a maximum timing interval of 256us.
- To measure longer periods of time, we can use:
 - Pre-scaler
 - Interrupts



Source: <https://learn.mikroe.com/ebooks/picmicrocontrollersprogramminginassembly/front-matter/introduction-to-the-world-of-microcontrollers/>

CE2007

Microprocessor-based System Design

Signals and Interfacing

Part 2

Topics Covered

- Polling vs. Interrupt
- Nested Vectored Interrupt Controller (NVIC)
- What Happens during an Interrupt?
- Interrupt Vector
- Interrupt Service Routine (ISR)
- Timer Interrupt
- Direct Memory Access (DMA)

Polling vs. Interrupt

Polling vs. Interrupt

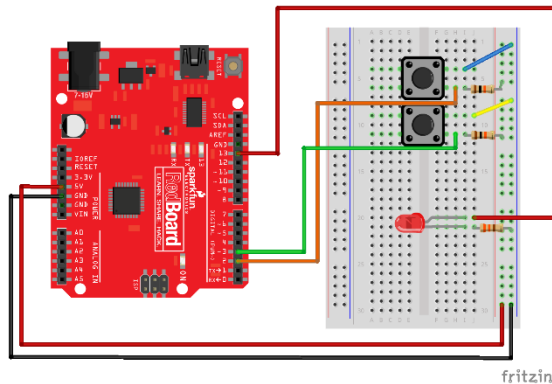
- Polling
 - Analogy: Pick up the phone every few seconds to check if someone is calling you.
 - Simple but inefficient.
- Interrupt
 - Analogy: Pick up the phone only when it rings, meanwhile do something else.
 - More efficient than polling.

```
// Polling
while (1) {
    read_input_button;
    if (pressed)
        exit;
}
turn_on_LED;
```

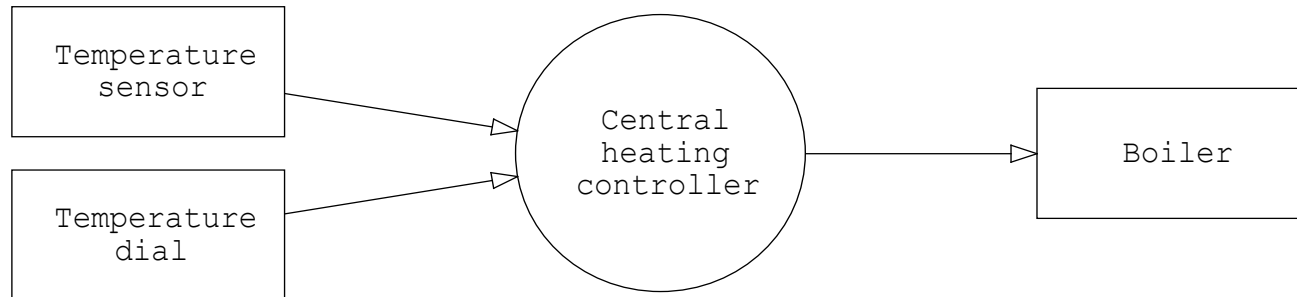
```
// Interrupt
main () {
    ...
}

Interrupt_handler() {
    turn_on_LED;
    exit;
}
```

Source:
<https://learn.sparkfun.com/tutorials/button-pad-hookup-guide>



Example of Polling



```
void main(void)
{
    /* Init the system */
    C_HEAT_Init();

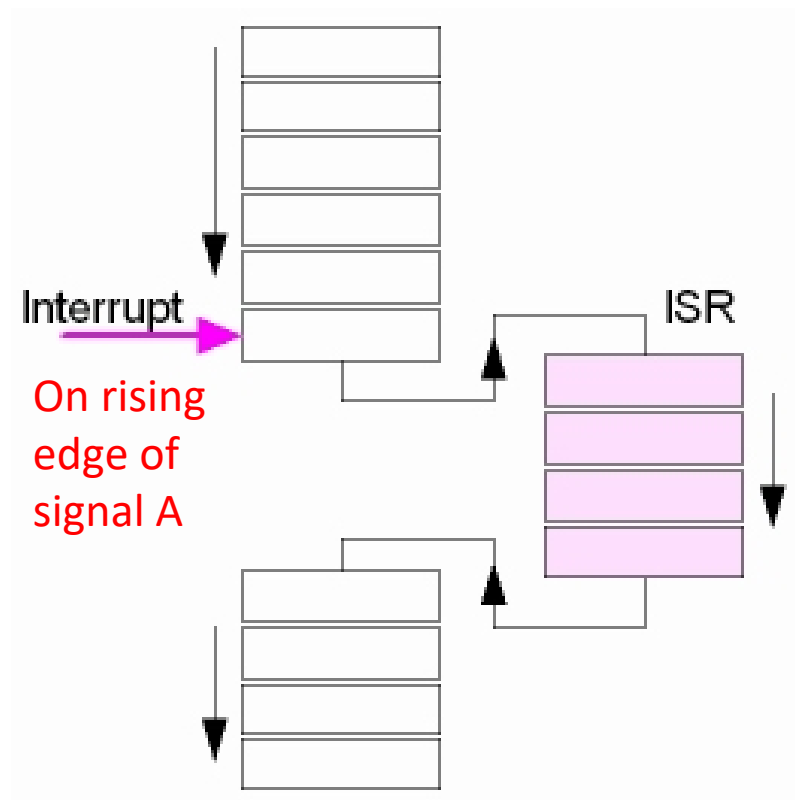
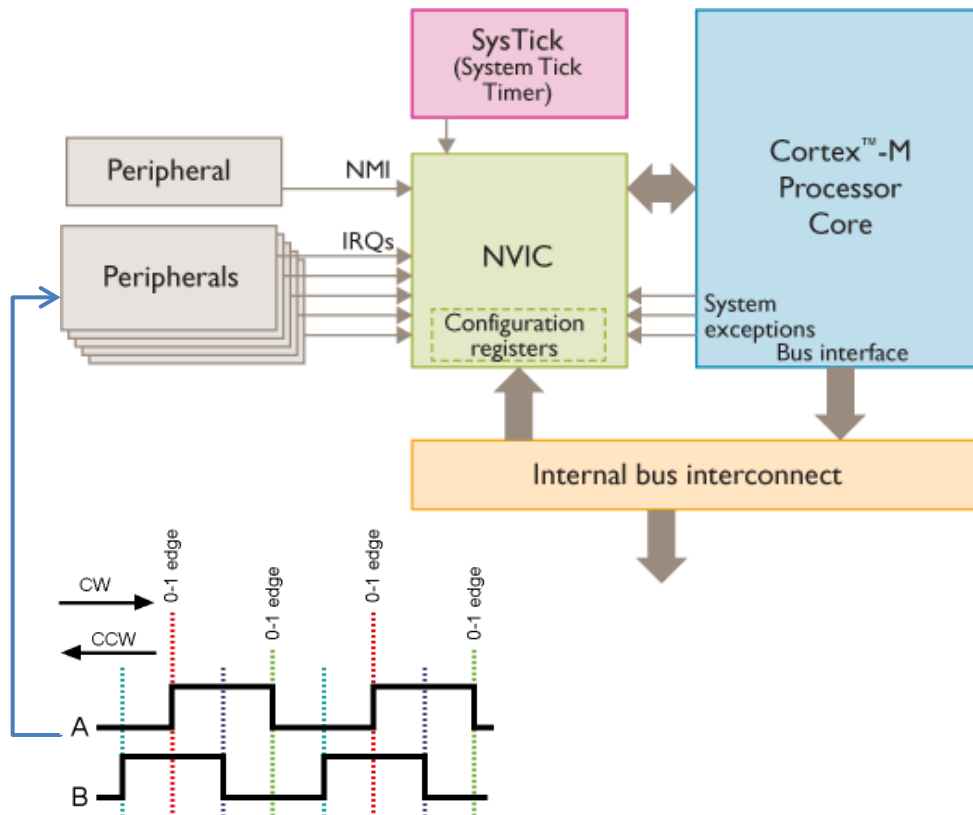
    while(1) /* 'for ever' (Super Loop) */
    {
        /* Find out what temperature the user requires
           (via the user interface) */
        C_HEAT_Get_Required_Temperature();

        /* Find out what the current room temperature is
           (via temperature sensor) */
        C_HEAT_Get_Actual_Temperature();

        /* Adjust the gas burner, as required */
        C_HEAT_Control_Boiler();
    }
}
```

Interfacing Using Interrupt

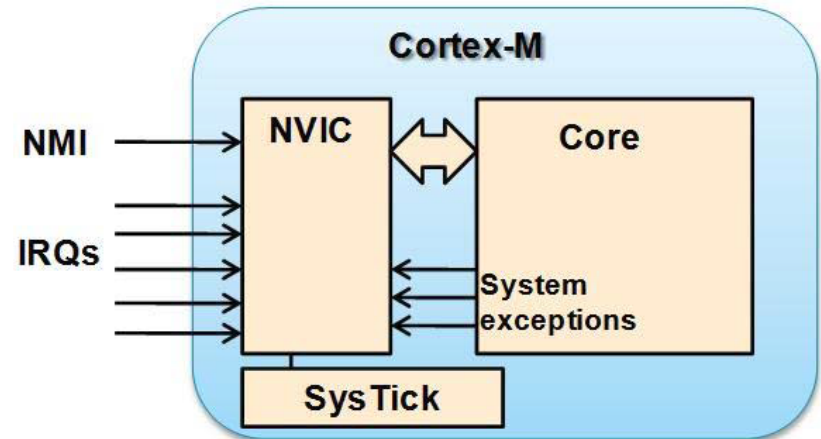
- Use interrupt to detect rising edge of signal A
- Write an interrupt service routine (ISR) to increment/decrement counter
 - ISR is a block of code that the microcontroller will jump to when the interrupt is triggered



Nested Vector Interrupt Controller (NVIC)

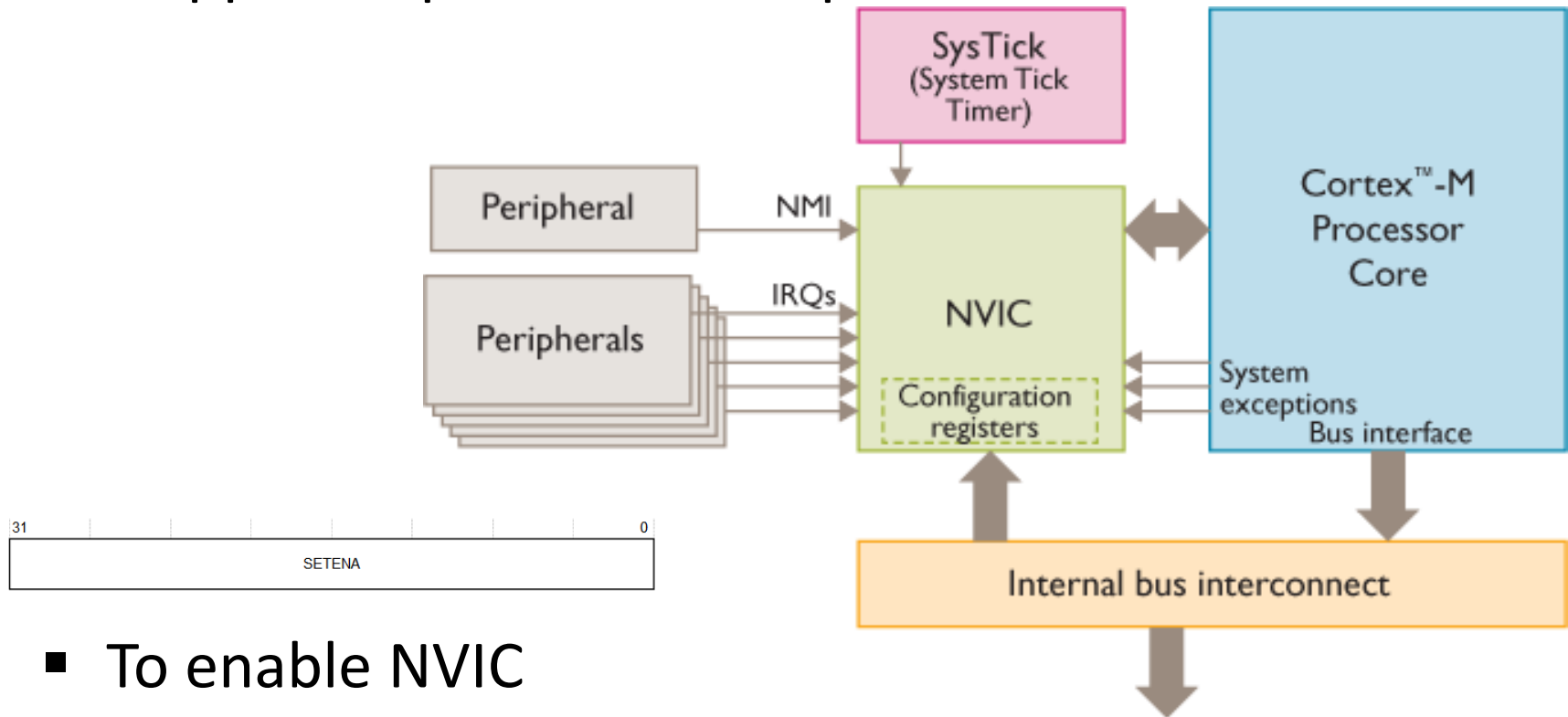
Nested Vector Interrupt Controller

- NVIC in the Cortex-M family is an interrupt controller with flexible interrupt priority management.
- Cortex-M0 and Cortex-M1
 - NVIC supports up to 32 external interrupts (IRQs).
- Cortex-M3 and Cortex-M4
 - NVIC supports up to 240 external interrupts (IRQs).
- NVIC also supports exceptions:
 - Exception input from SysTick timer.
 - System exceptions to support OS operations.
 - Non-Maskable Interrupt (NMI).



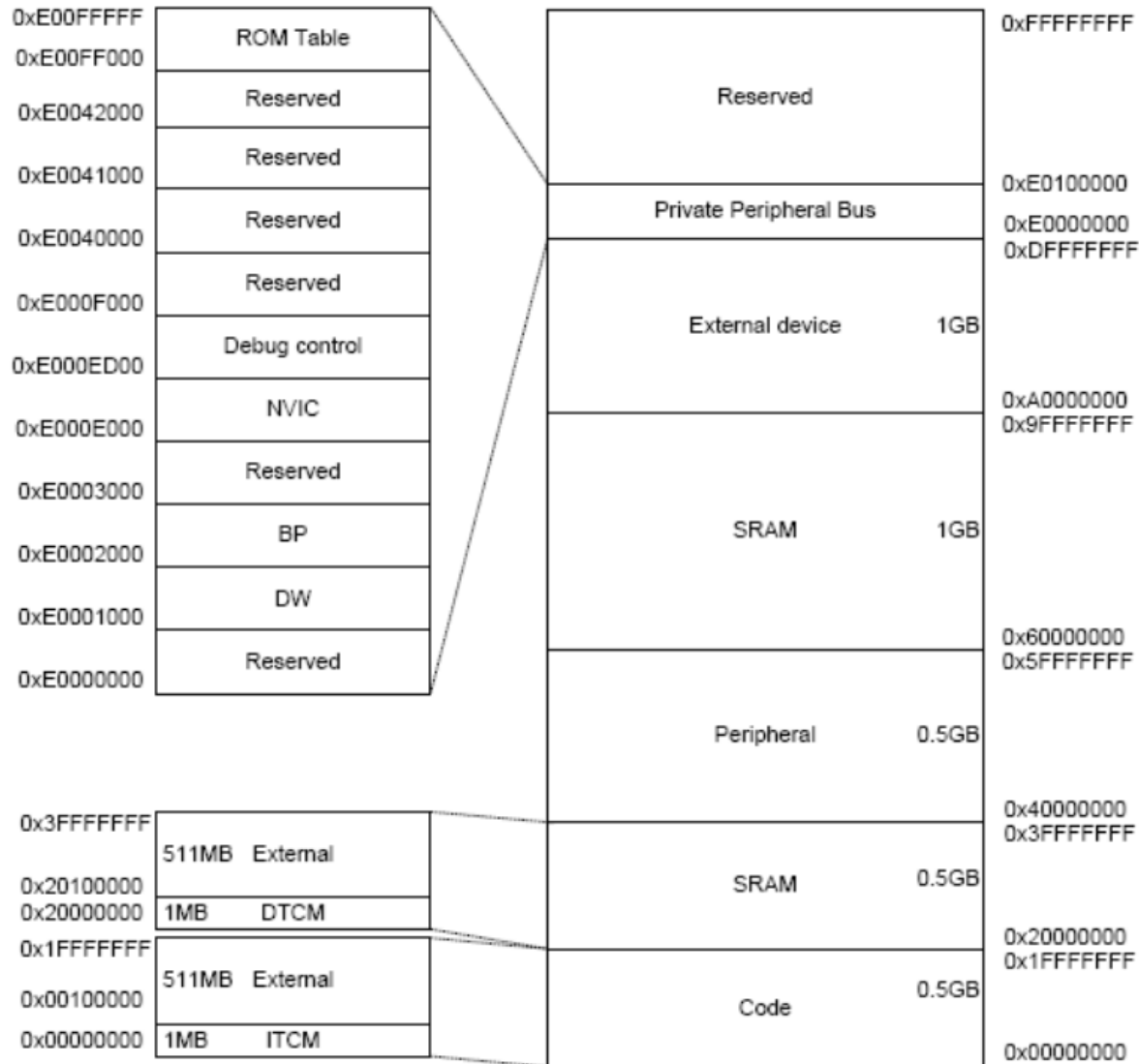
ARM Nested Vectored Interrupt Controller (NVIC)

- Tightly coupled to the Cortex-M processor core to facilitate low latency exception processing
- Supports up to 32 interrupts



- To enable NVIC
 - Set Enable register memory-mapped at address [0xE000E100](#)
 - Set interrupt priority (optional)

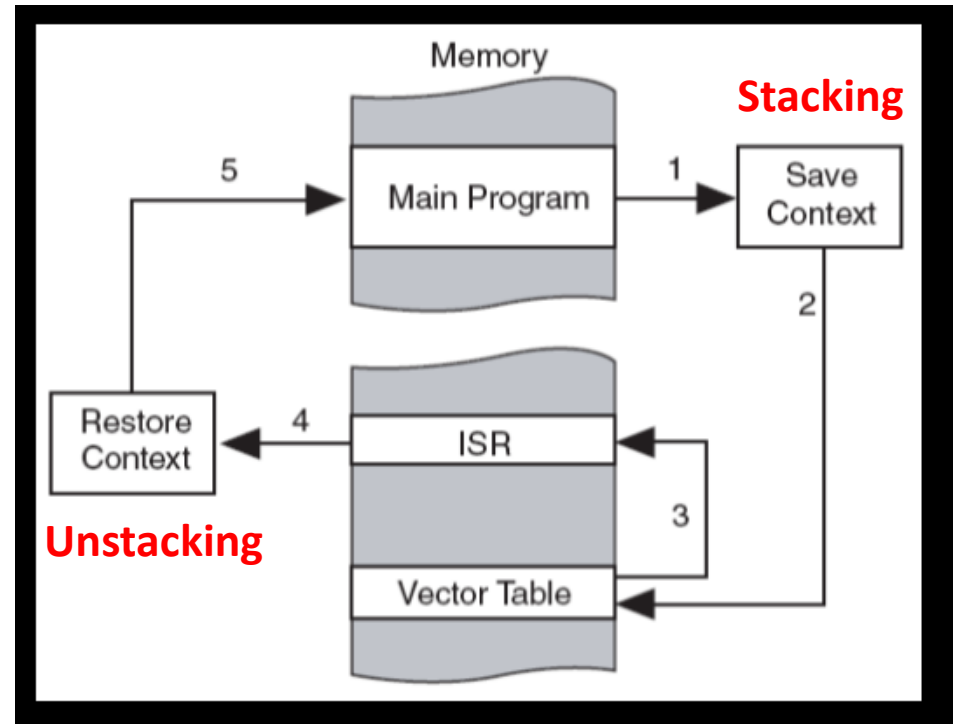
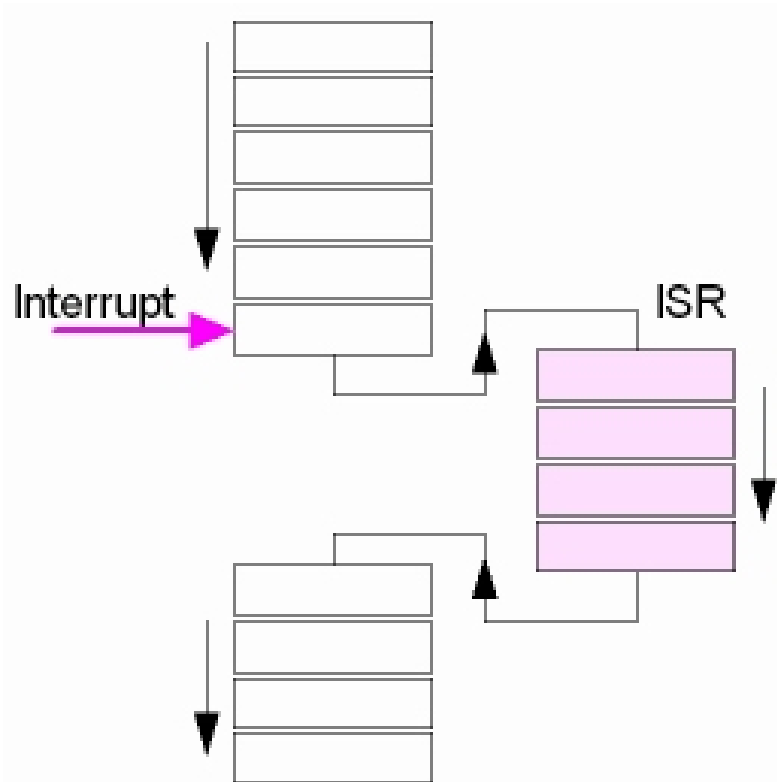
Cortex-M1 Memory Map



What Happens During Interrupt?

What Happens During Interrupt?

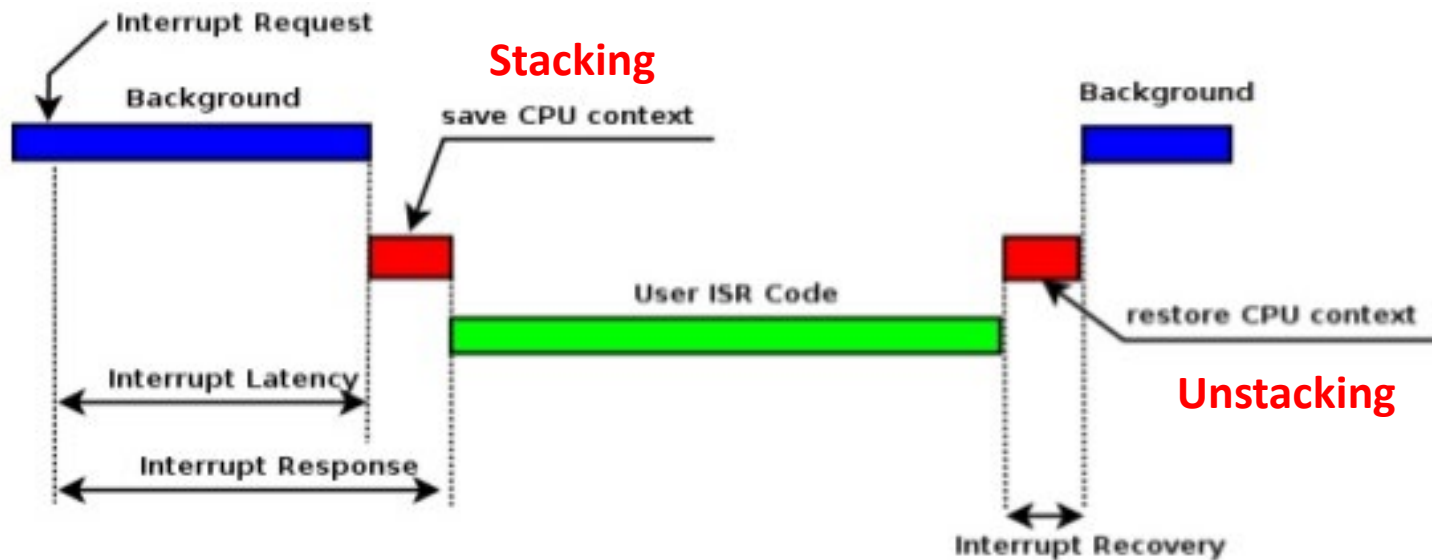
- Program suspended and interrupt service routine (ISR) is invoked



Source: <http://cnx.org/content/m45965/latest/>

Interrupt Timing

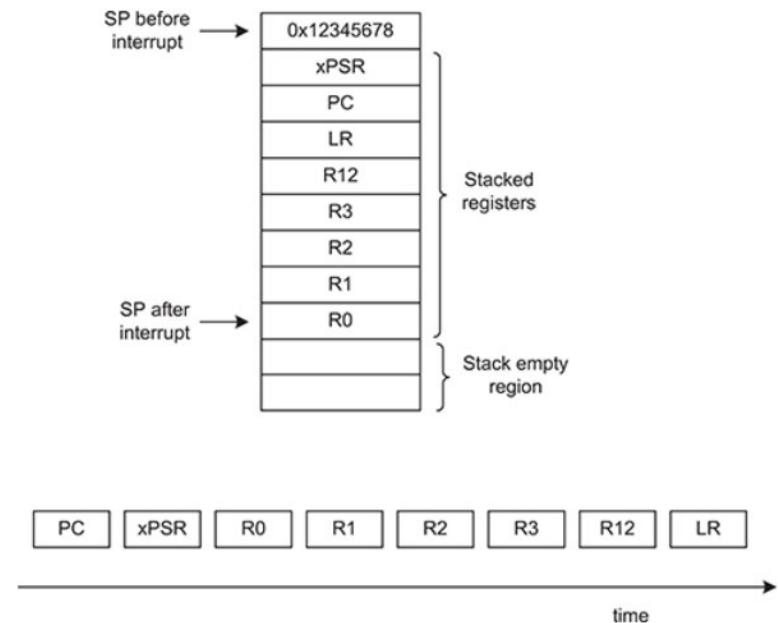
- **Interrupt Latency:** Time required for a processor to respond to an interrupt request.
 - Typically, processors only start to respond to the interrupt request when current instruction completes execution.
- **Interrupt Response:** Time required for interrupt to be serviced by ISR after an interrupt request.



Source: <https://www.slideshare.net/MostafaElkoumy/embedded-c-workshop>

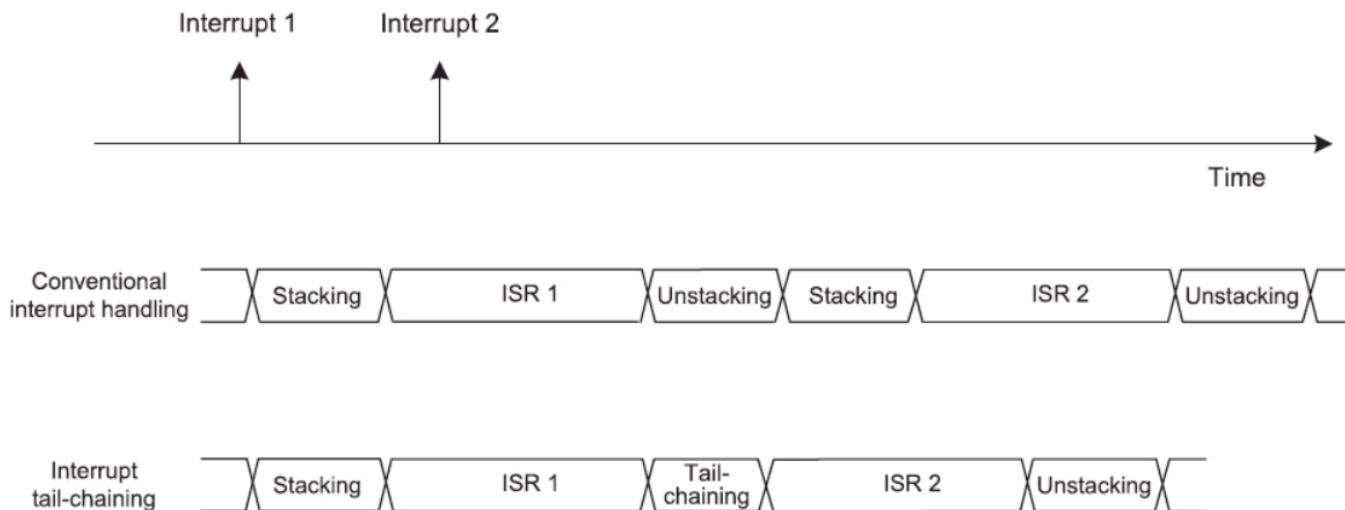
Register Stacking and Unstacking

- Processor automatically stacks its state before entering the interrupt service routine and unstacks the system state while exiting from the ISR.
- Cortex-M has built-in hardware capability of register stacking and unstacking
 - No PUSH and POP instructions are executed while stacking and unstacking the registers, which provides low latency interrupt handling.
- Any other general purpose register that can be modified by the exception handler should be saved on the stack by the user.



Tail Chaining

- The Cortex-M processor uses tail chaining to reduce overhead during interrupt handling.
- Tail Chaining:
 - When an ISR is completed, and another ISR is waiting to be served, processor executes new ISR by skipping some stacking/unstacking operations.

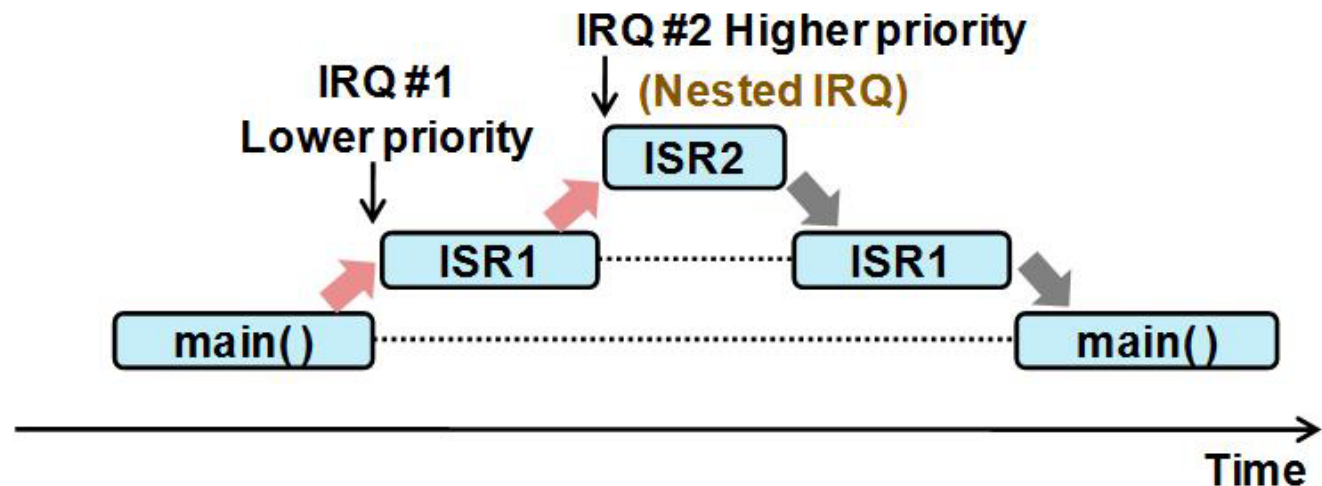


Source: Tahir, Muhammad. *ARM Microprocessor Systems*. CRC Press, 20170217

Interrupt Priority

Interrupt Priority

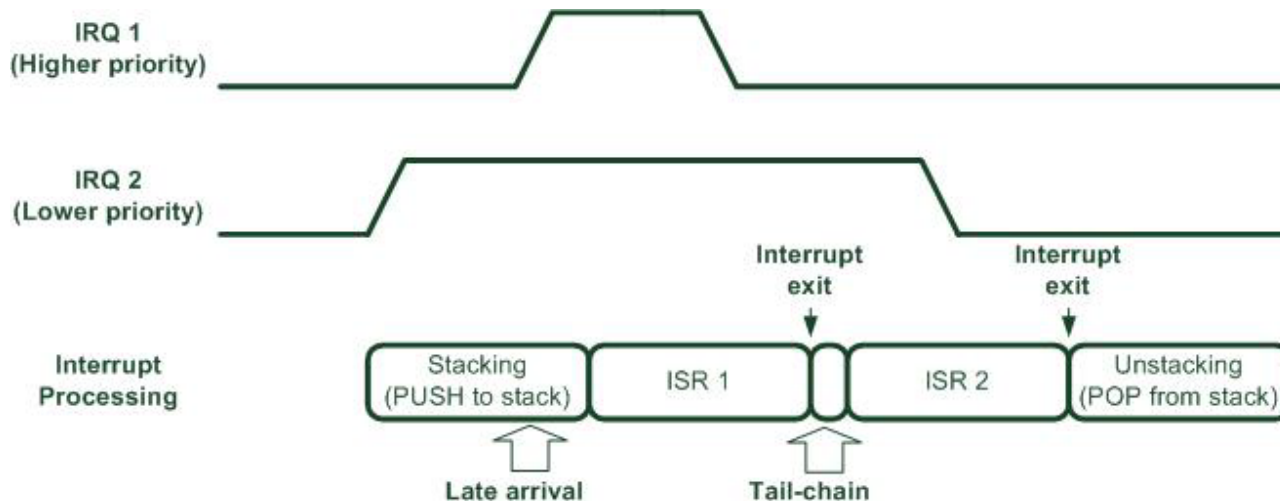
- NVIC allows multiple interrupt priorities (along with fixed priorities)
- Cortex-M0 and Cortex-M1
 - 4 programmable priority levels
- Cortex-M3 and Cortex-M4
 - 8 to 256 programmable priority levels
- The execution of an ISR corresponding to a low priority interrupt might be suspended due to the occurrence of a high priority interrupt.



Tail Chaining

■ Late Arrival:

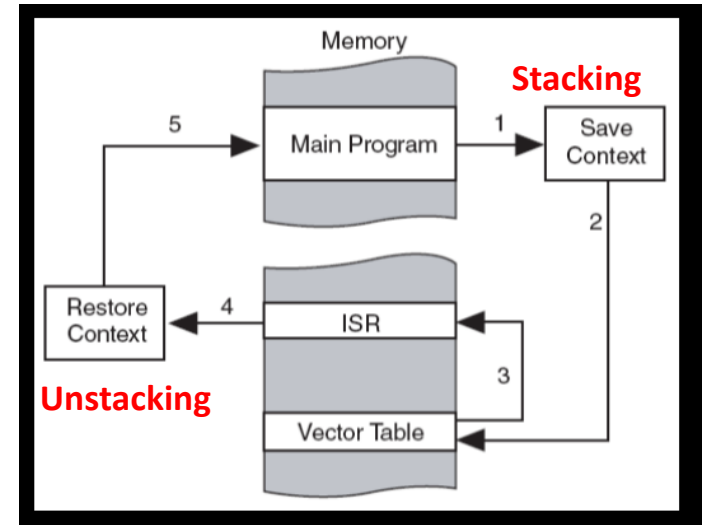
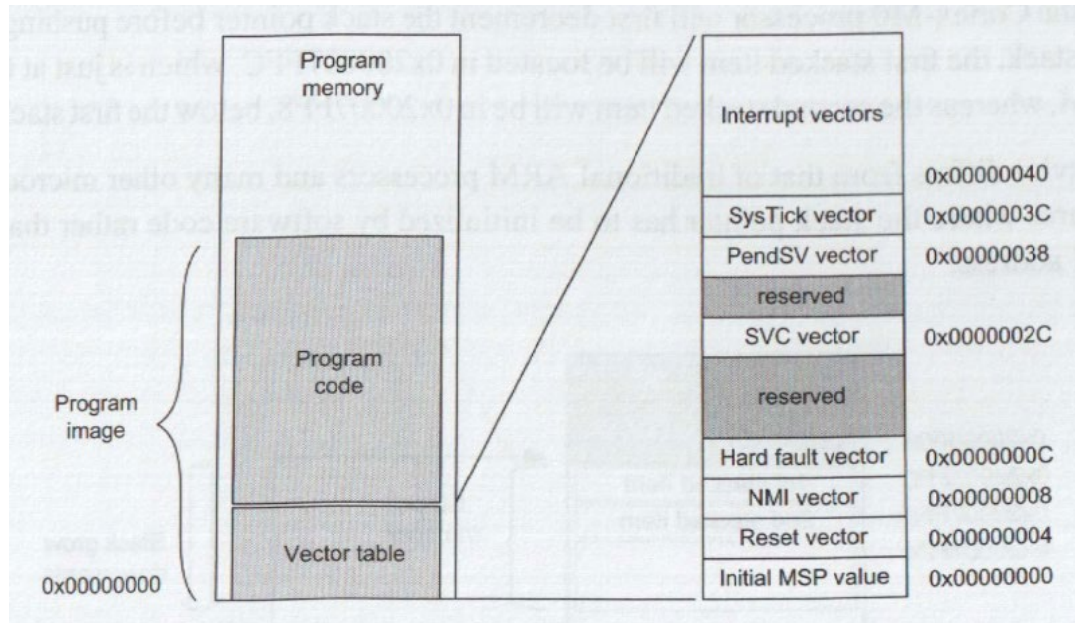
- High priority interrupt arrives during stacking stage of low priority interrupt.
- High priority interrupt are serviced first.
- Avoid stacking operation before servicing low priority interrupt.



Interrupt Vector

Interrupt Vector

- Upon interrupt, the Cortex-M1 needs to know the address of the ISR
- The address (vector) of the Interrupt Handler is stored in the **vector table**, which is located at address **0x00000000**
- During context switch, the interrupt vector is loaded into PC to execute the ISR
 - Need to ensure that PC is loaded with return address after completion of ISR



Example of Start-Up code

```
Stack_Size      EQU      0x00000200
                  AREA    STACK, NOINIT, READWRITE, ALIGN=3
Stack_Mem       SPACE    Stack_Size
__initial_sp

Heap_Size       EQU      0x00000400
                  AREA    HEAP, NOINIT, READWRITE, ALIGN=3
__heap_base
Heap_Mem        SPACE    Heap_Size
__heap_limit

; Vector Table Mapped to Address 0 at Reset
                  AREA    RESET, DATA, READONLY
                  EXPORT  __Vectors
__Vectors       DCD      __initial_sp          ;
                  DCD      Reset_Handler        ;
                  DCD      NMI_Handler          ;
                  DCD      HardFault_Handler    ;
                  :
Reset_Handler   PROC
                  EXPORT  Reset_Handler
                  IMPORT  __main
                  LDR      R0, =__main
                  BX       R0
                  ENDP
```

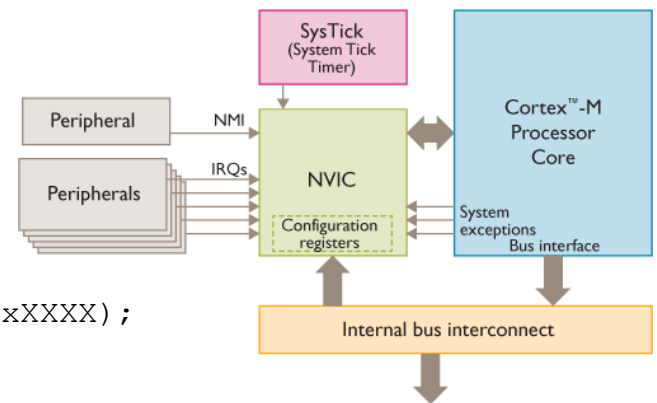
Interrupt vectors	
	0x00000040
SysTick vector	0x0000003C
PendSV vector	0x00000038
reserved	
SVC vector	0x0000002C
reserved	
Hard fault vector	0x0000000C
NMI vector	0x00000008
Reset vector	0x00000004
Initial MSP value	0x00000000

Example: ISR

```
// Timer's Interrupt service handler.
// Mapped to IRQ0 in the DE0-TAB platform
void timer_isr (void) __irq {
    // Acknowledge the interrupt
    PUT_UINT32(CM1_TIMER_BASE + CM1_TIMER_STATUS_OFFSET, 0xFFFF);
    timeout = 1;
}
```

; Vector Table Mapped to Address 0 at Reset

	AREA EXPORT	RESET, DATA, READONLY __Vectors	
__Vectors	DCD	__initial_sp	; Top of Stack
Handler	DCD	Reset_Handler	; Reset
	DCD	NMI_Handler	; NMI Handler
Handler	DCD	HardFault_Handler	; Hard Fault
	. . .		
Handler	DCD	SVC_Handler	; SVCall
	DCD	0	; Reserved
	DCD	0	; Reserved
Handler	DCD	PendSV_Handler	; PendSV
	DCD	SysTick_Handler	; SysTick
Handler	DCD	IRQ0_Handler	; IRQ0 Handler
	. . .		
IRQ0_Handler	PROC		
	IMPORT	timer_isr	
	LDR	R0, =timer_isr	
	BX	R0	
	ENDP		



Interrupt vectors	
SysTick vector	0x00000040
PendSV vector	0x0000003C
reserved	0x00000038
SVC vector	0x0000002C
reserved	
Hard fault vector	0x0000000C
NMI vector	0x00000008
Reset vector	0x00000004
Initial MSP value	0x00000000

Startup.s

Interrupt Service Routine (ISR)

Tutorial 5: Question 1

1. A microprocessor is interfaced to a Keypad as shown in the figure below.

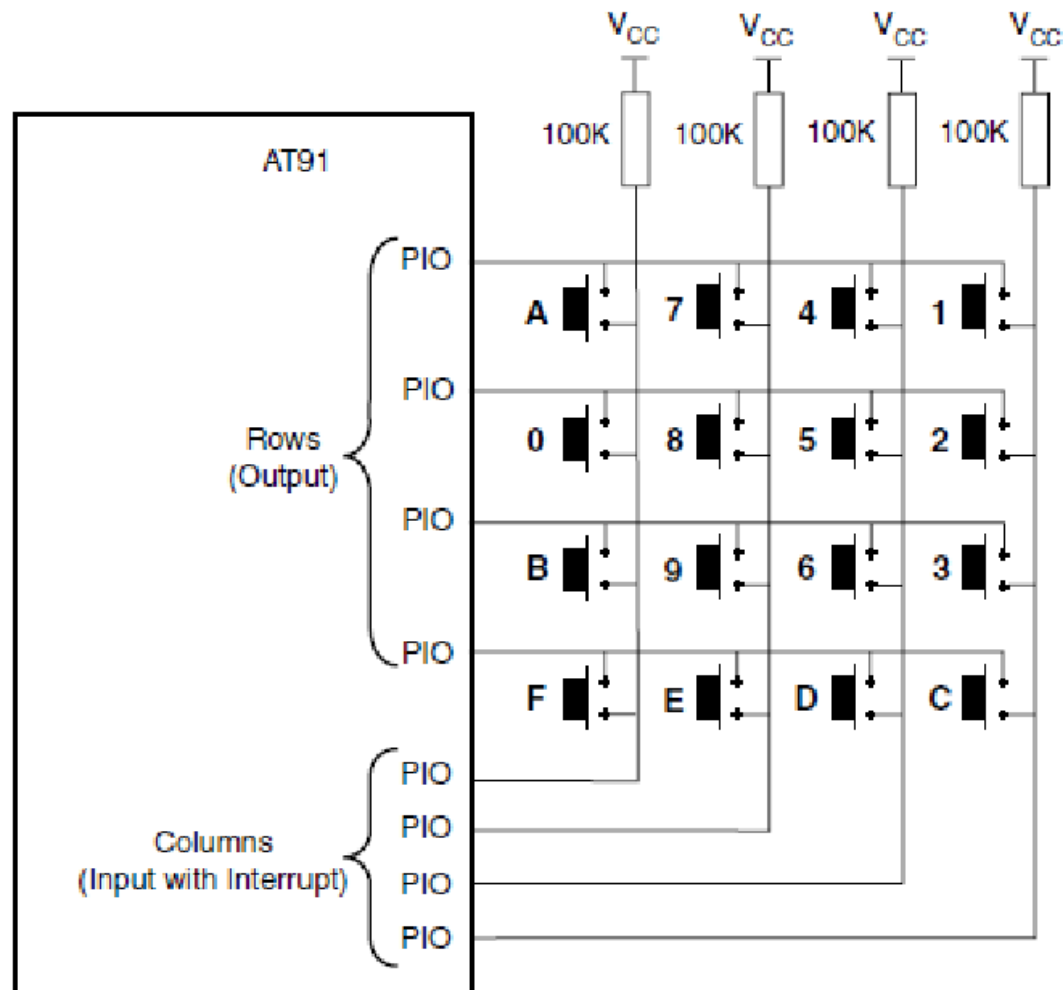
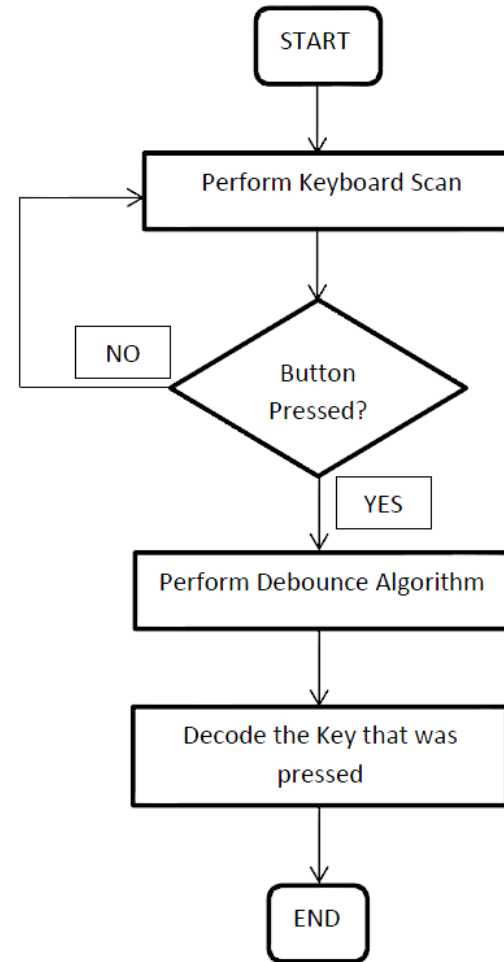
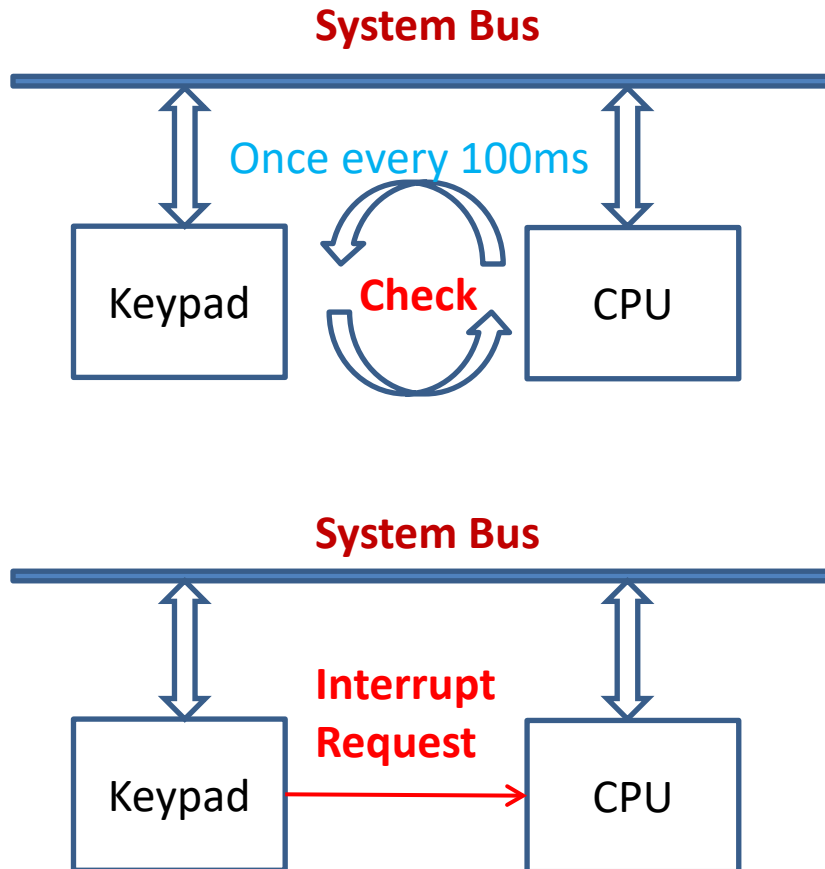


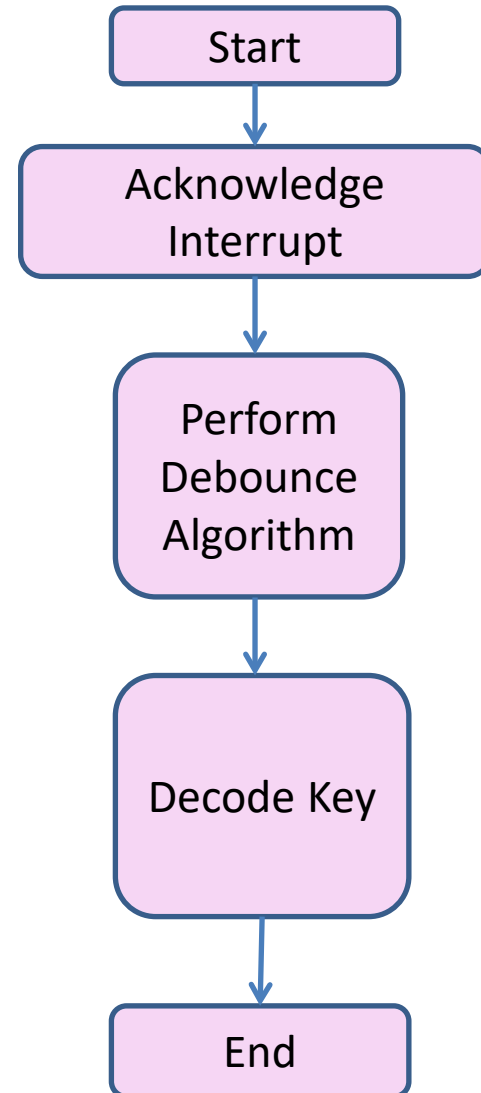
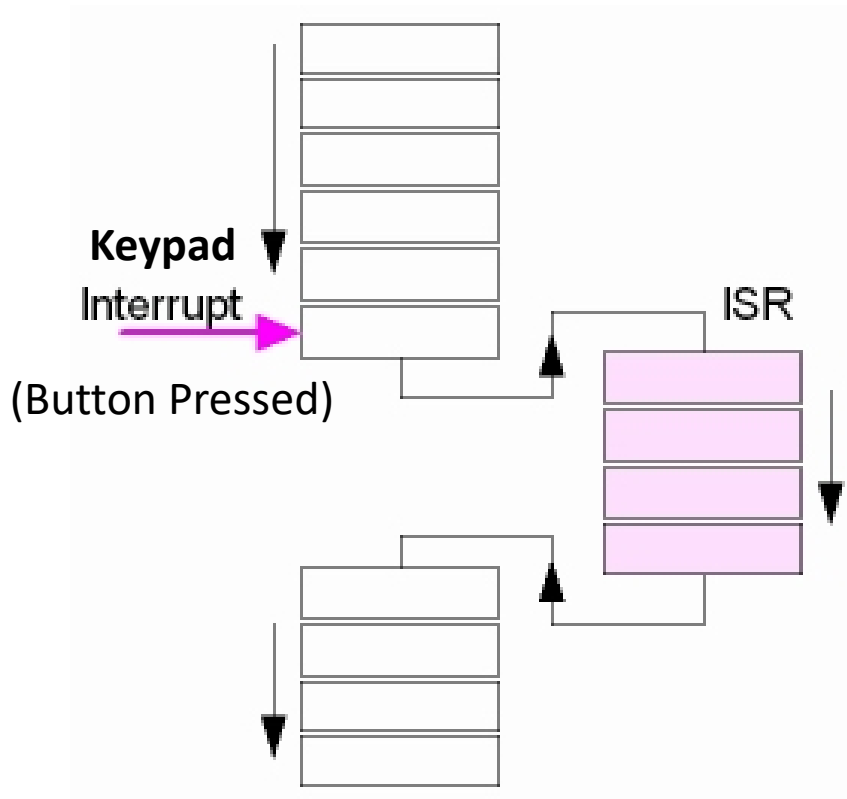
Figure 1: Microprocessor interfaced to 4x4 Keypad

Tutorial 5: Question 1

Design a flowchart for the software interface using the “Software Polling” approach.



Interrupt Service Routine



Interrupt Service Routine (ISR)

Interrupt Service Routine (ISR)

- Need to do the following:
 - Acknowledge the interrupt by clearing the trigger flag in device
 - Failure to acknowledge the interrupt may cause a system crash
 - Service the interrupt
- Good practice when writing an ISR
 - Avoid delay loop (keep ISR short)
 - Execute time of ISR should be shorter than rate of interrupt triggers

Timer Interrupt

Accessing the Timer

- Timer is memory-mapped at address 0xA0002000 (Base)

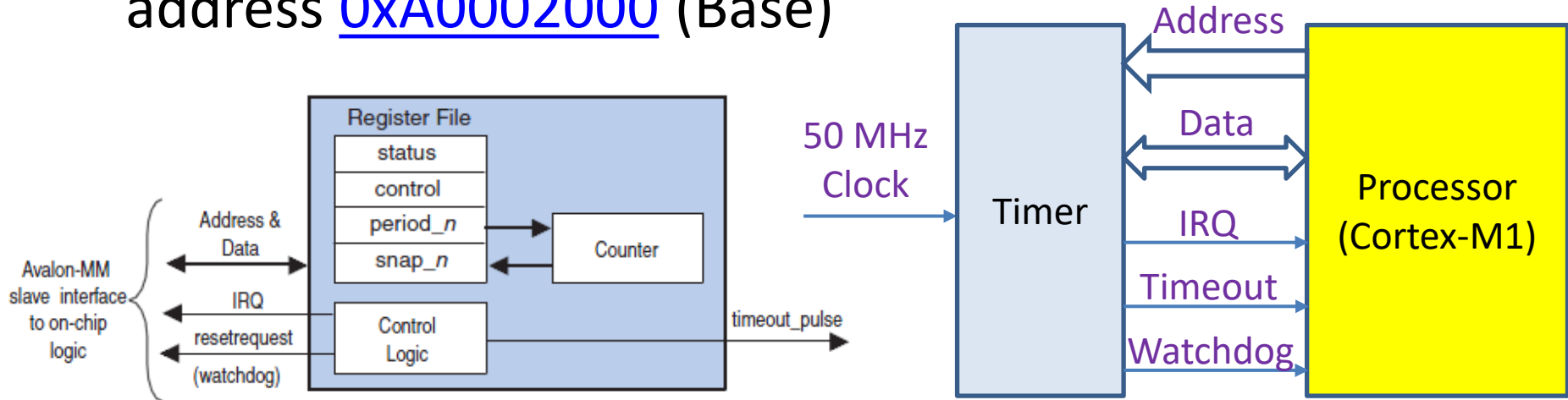


Table 28–3. Register Map—32-bit Timer

Offset	Name	R/W	Description of Bits						
			15	...	4	3	2	1	0
0	status	RW	(1)						TO
1	control	RW	(1)			STOP	START	CONT	ITO
2	periodl	RW	Timeout Period – 1 (bits [15:0])						
3	periodh	RW	Timeout Period – 1 (bits [31:16])						
4	snapl	RW	Counter Snapshot (bits [15:0])						
5	snaph	RW	Counter Snapshot (bits [31:16])						

Note to Table 28–3:

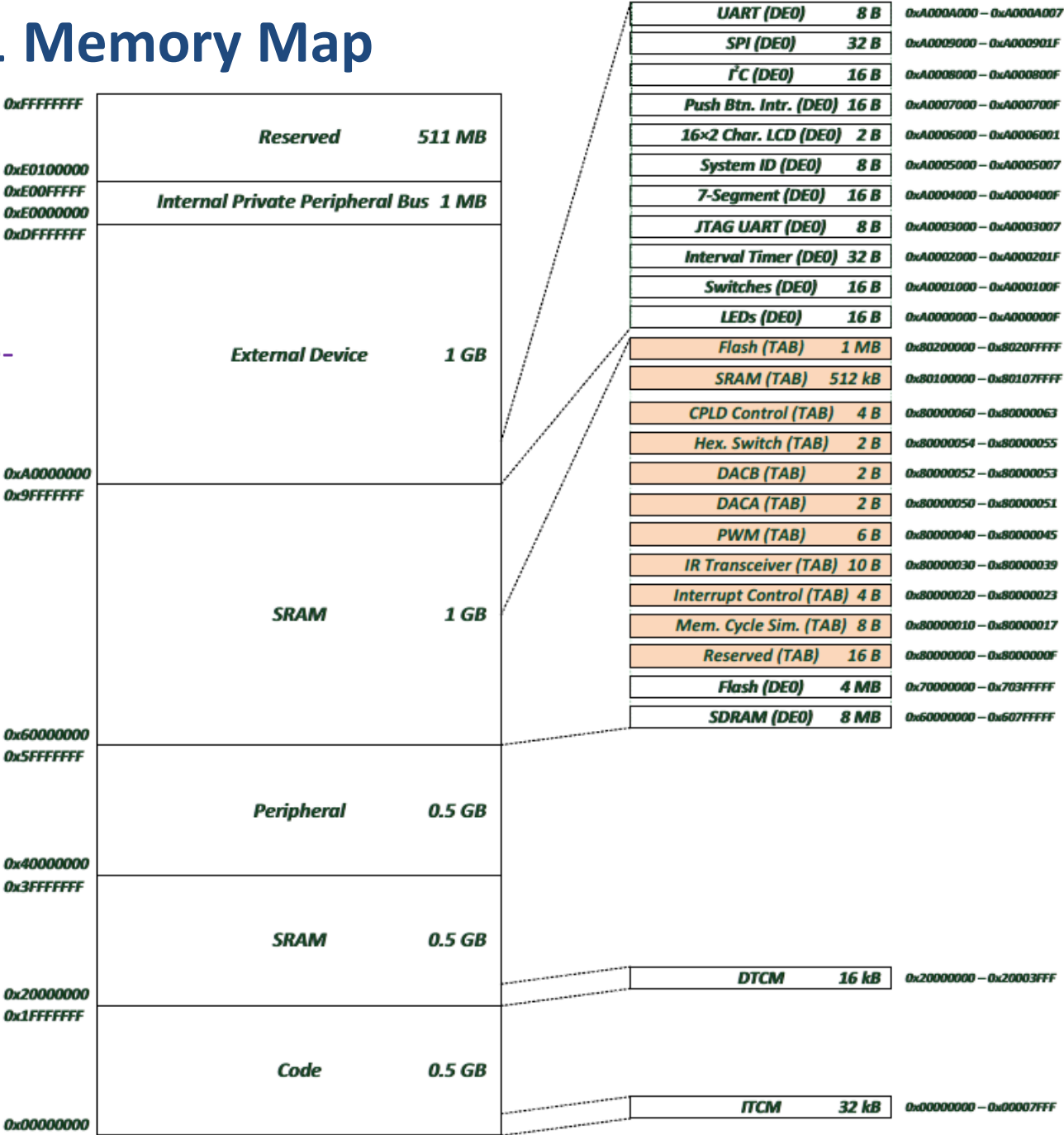
(1) Reserved. Read values are undefined. Write zero.

Source: Altera: Embedded Peripherals IP User Guide

DE0 Cortex-M1 Memory Map

Note that SDRAM and Flash memory on DE0 board, Flash and other components on TAB board are mapped to pre-defined SRAM memory region of Cortex-M1 processor at address range **0x60000000 – 0x9FFFFFFF**

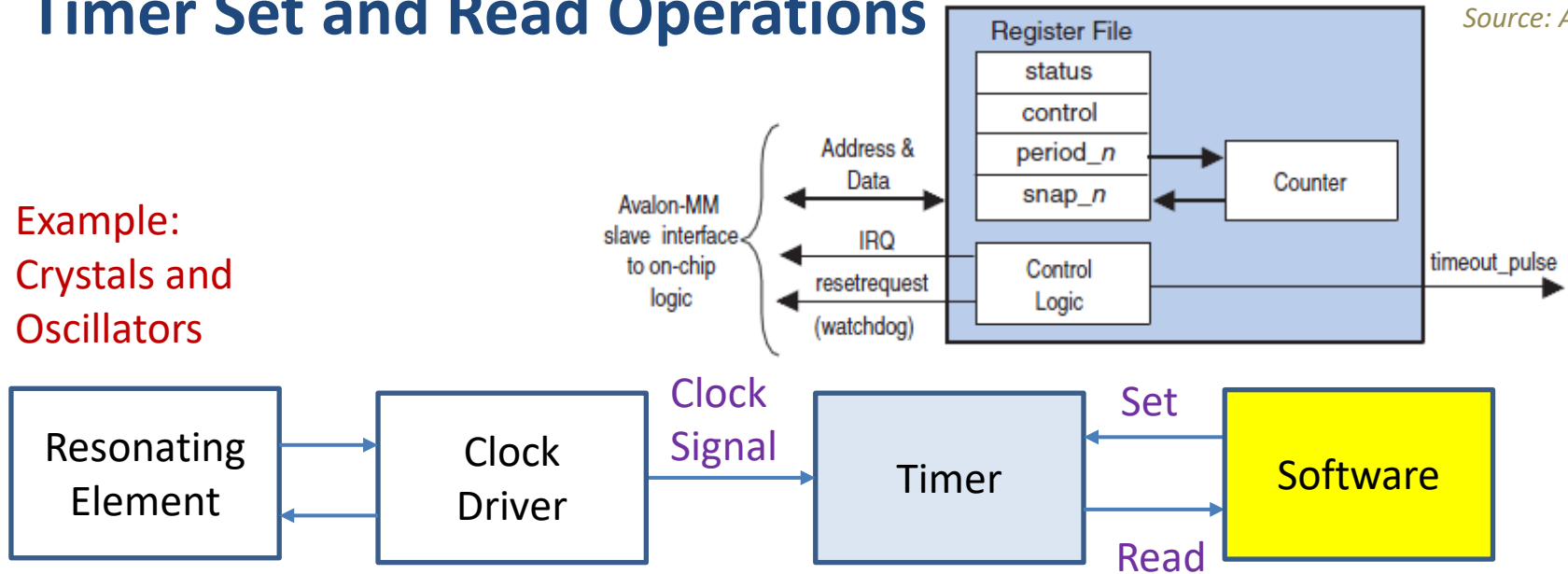
Both ITCM and DTCM utilize on-chip memory residing in the **FPGA**



Timer Set and Read Operations

Source: Altera

Example:
Crystals and
Oscillators



■ Set Operations

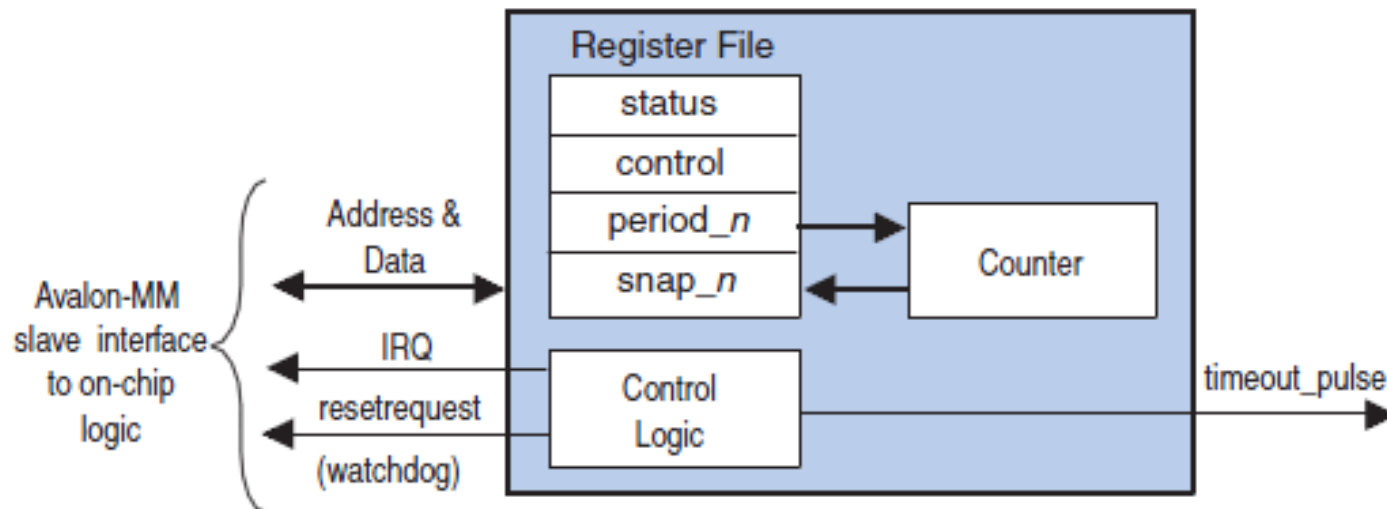
- Specify Timer Period
 - Period_n
- Enable/Disable Interrupt Request (IRQ)
 - Control register
- Start/Stop Timer
 - Control register

■ Read Operations

- Current timer activity (e.g. is counter running or has it time-out?)
 - Status register
- Counter value
 - Snap_n

Timer Operations

- Counter counts-down to 0
- When counter reaches 0
 - Counter is automatically reloaded with timer period
 - Status register is updated
 - Timeout pulse is asserted (optional)
 - If IRQ is enabled, an IRQ pulse is generated
 - Watchdog output is asserted (optional)



How Do We Know Timeout has Occurred?

- Polling
 - Monitor timeout pulse
 - Read status register
 - TO bit is set
- Timer Interrupt

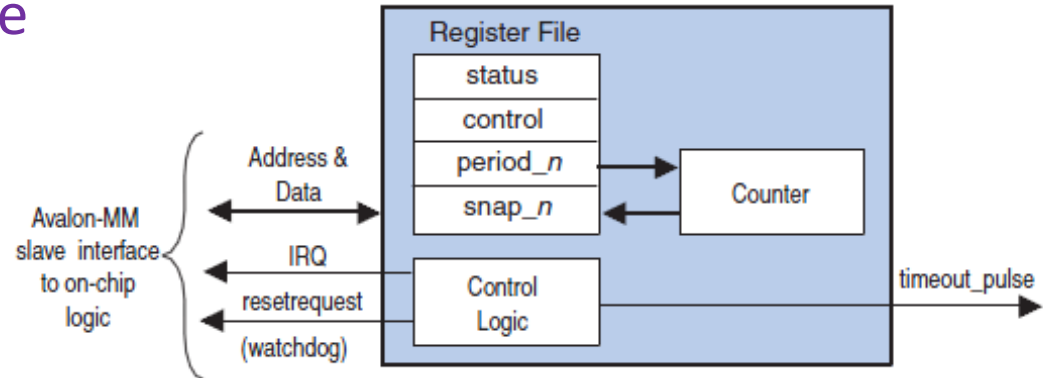


Table 28–3. Register Map—32-bit Timer

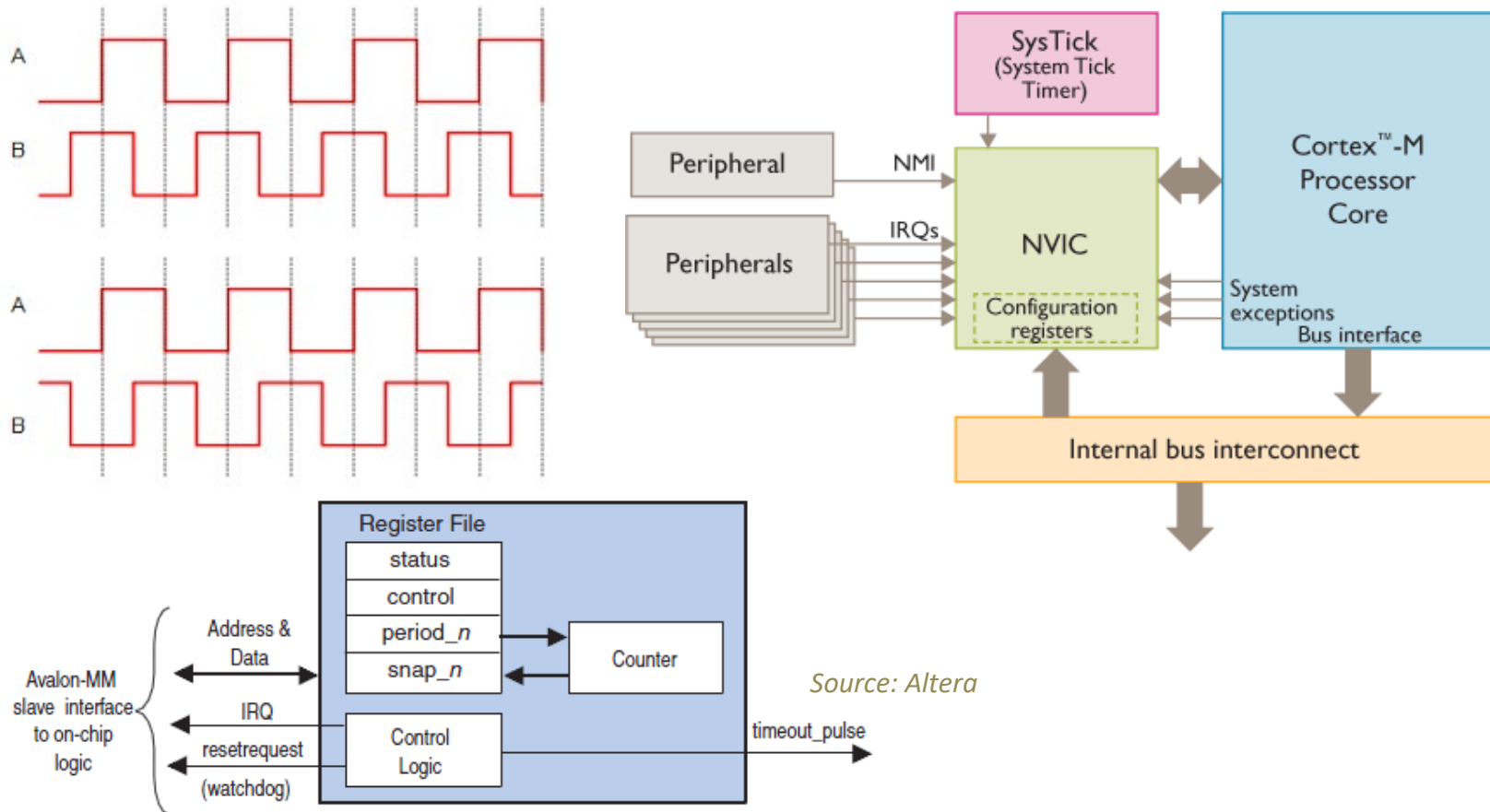
Offset	Name	R/W	Description of Bits						
			15	...	4	3	2	1	0
0	status	RW	(1)						TO
1	control	RW	(1)			STOP	START	CONT	ITO
2	periodl	RW	Timeout Period – 1 (bits [15:0])						
3	periodh	RW	Timeout Period – 1 (bits [31:16])						
4	snapl	RW	Counter Snapshot (bits [15:0])						
5	snaph	RW	Counter Snapshot (bits [31:16])						

Note to Table 28–3:

(1) Reserved. Read values are undefined. Write zero.

Timer Interrupts

- Timer Interrupts are useful for:
 - Measuring an incoming signal at equally spaced intervals (constant sampling frequency)
 - Sending out a signal of a specific frequency
 - Periodically checking for incoming serial data

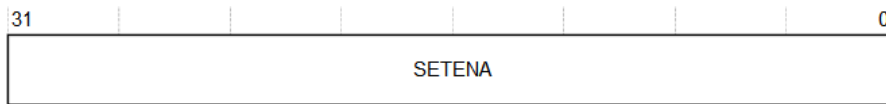
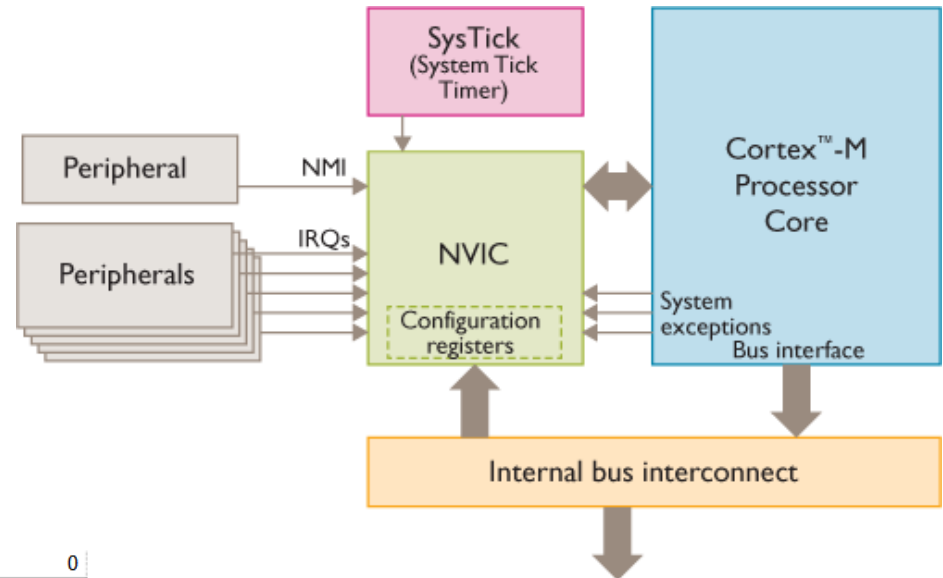


Steps for Setting Up Timers with Interrupt

1. IRQ must be connected to Nested Vectored Interrupt Controller (NVIC) of Cortex-M1

2. Enable NVIC

- Set Enable register memory-mapped at address 0xE000E100
- Set interrupt priority (optional)

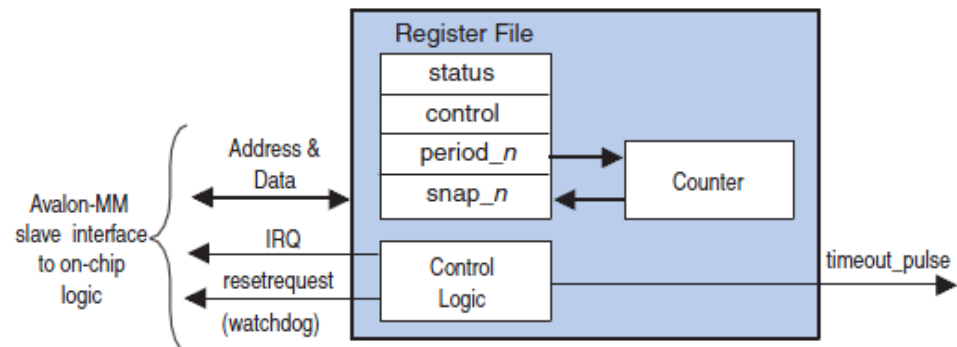


3. Enable IRQ in timer

- Control register

4. Set timer period

- Period_n register



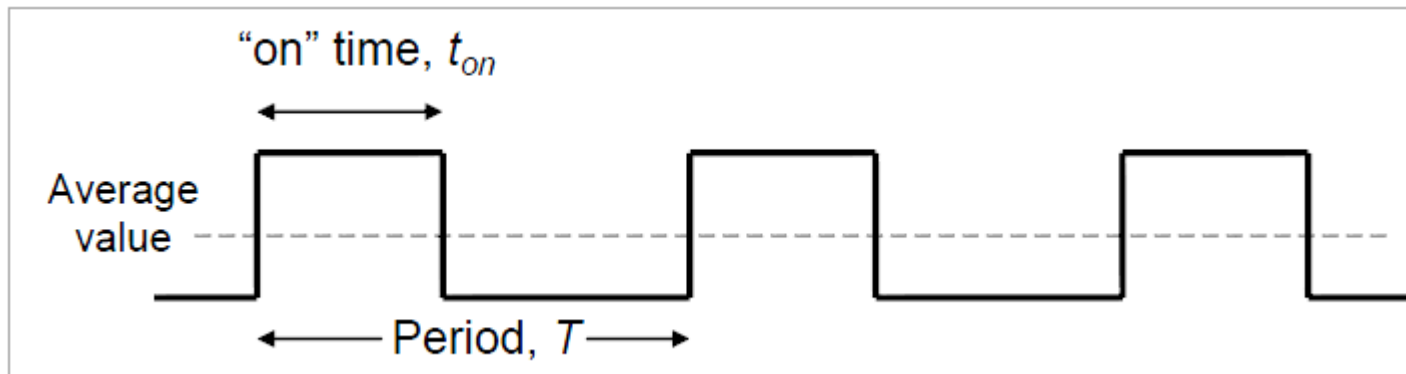
Tutorial 4: Real-Time Clock

- You are required to implement an Interrupt Service Routine (ISR) that computes the seconds of a real-time clock by making use of the timer available on a microcontroller.
- The timer is driven by a 50 MHz clock. The ISR must be triggered on a repetitive interrupt from the timer every 10ms.
 - Calculate the hexadecimal value that must be written to the timer register to set the timer period of 10ms.
 - Draw the flow-chart of the ISR and state any assumptions made.

Pulse Width Modulation (PWM)

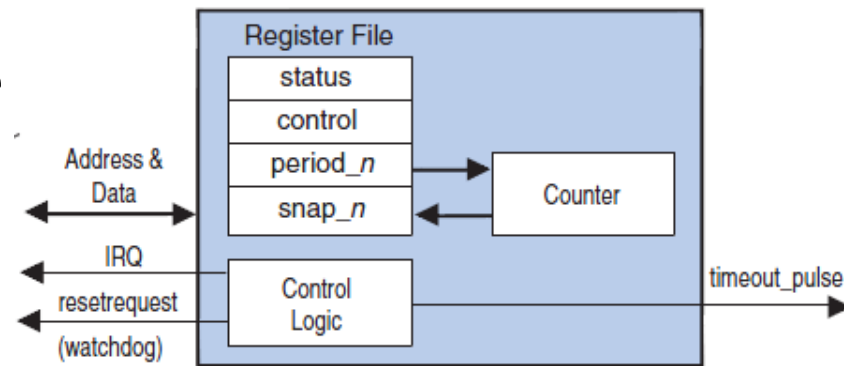
- PWM is a simple method of using a rectangular digital waveform to control an analogue variable
- The period is normally kept constant, and the pulse width, or “on” time is varied
- The duty cycle is the proportion of time that the pulse is ‘on’ or ‘high’, and is expressed as a percentage:

$$\text{duty cycle} = 100\% * (\text{pulse on time}) / (\text{pulse period})$$



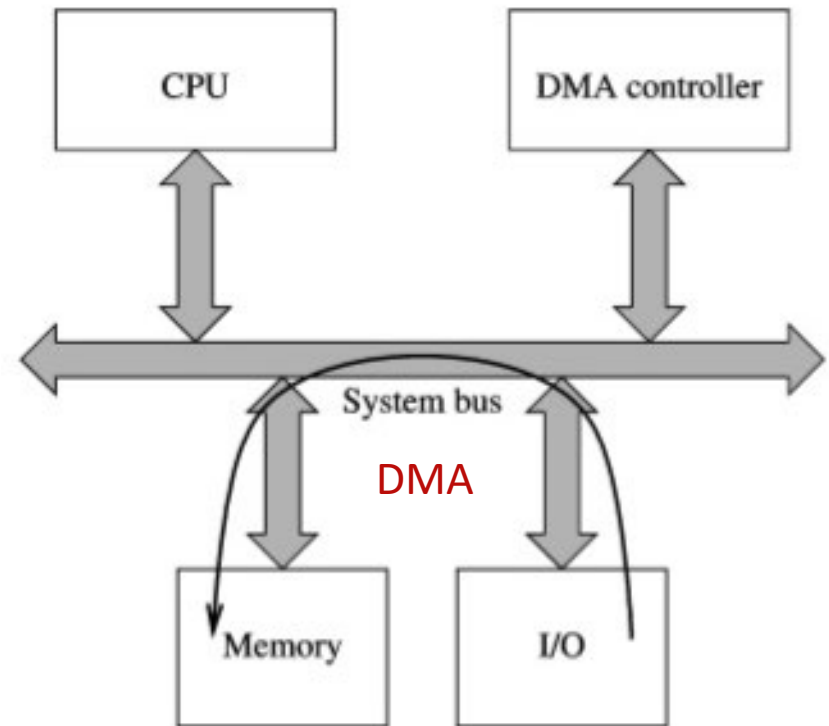
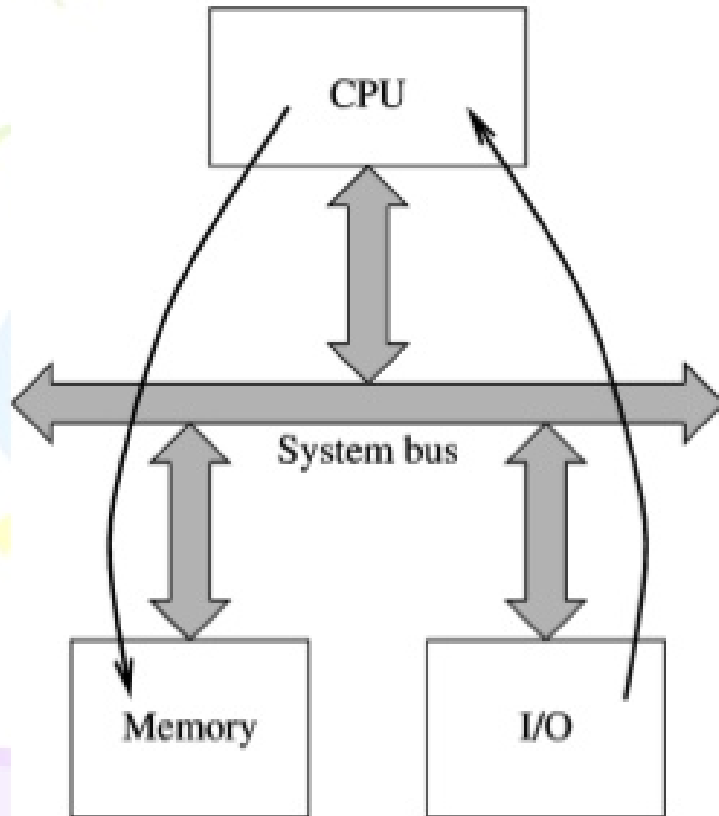
Watchdog Timer

- Embedded devices usually operate in total isolation and are not accessible to an operator
- When hardware/software anomalies occur, it may not be possible to manually reset the device when it “hangs”
 - This can result in damaged hardware or loss of life and incur significant cost impact
- A watchdog timer is a hardware timing device that triggers a system reset after a designated amount of time has elapsed
- To avoid a undesired reset, an application must periodically reset the watchdog timer before this interval elapses



Direct Memory Access (DMA)

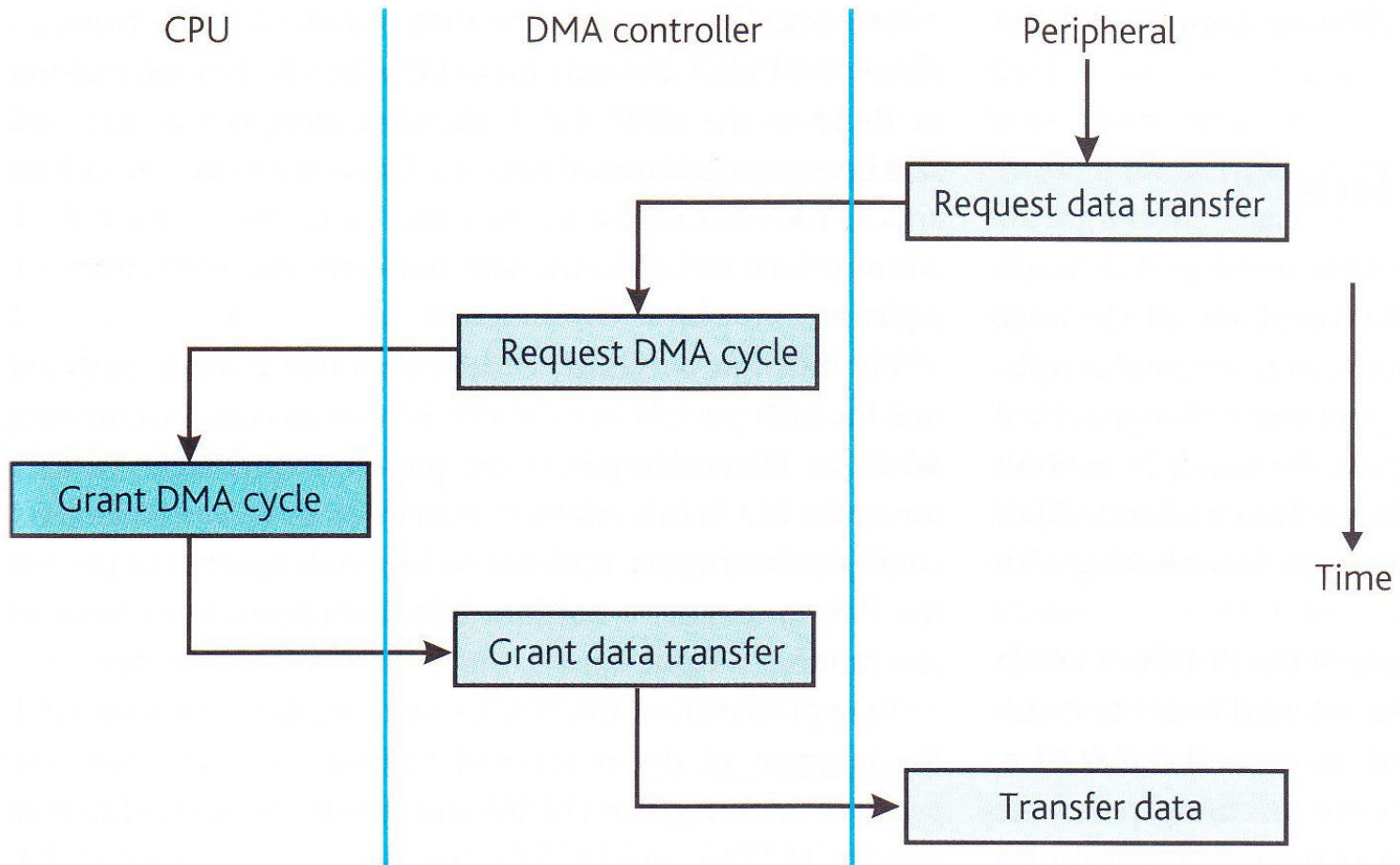
DMA I/O Accesses



Direct Memory Access (DMA)

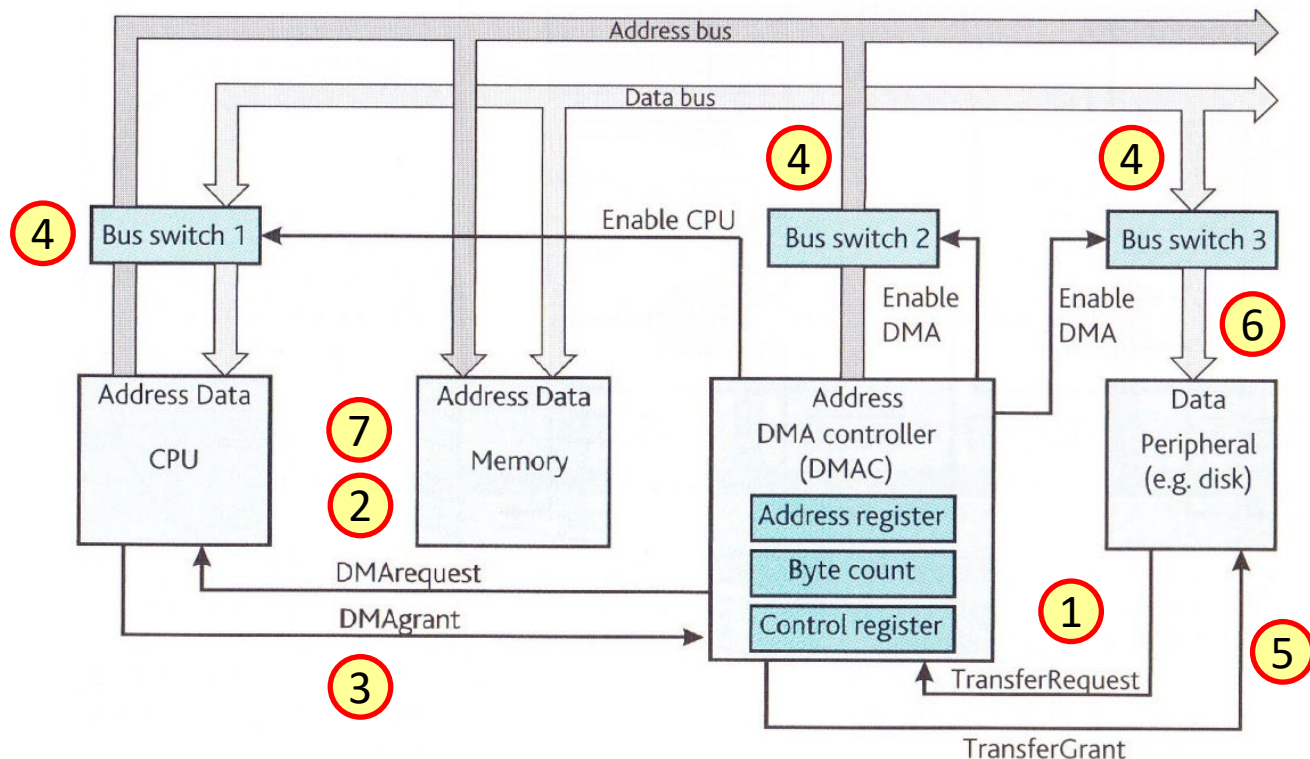
- Direct Memory Access (DMA) moves data between a peripheral and the CPU's memory without the direct intervention of the CPU itself.
- Provides the fastest possible means of transferring data between an interface and memory.
- It requires no CPU overhead and leaves the CPU free to do useful work.
- Helps to increase data throughput.

DMA Protocol



DMA I/O Access

- During **normal** operation of the computer, bus switch 1 is closed and bus switches 2 and 3 are open.
- The CPU control the buses, providing an address on the address bus and reading and writing data from memory via the data bus.

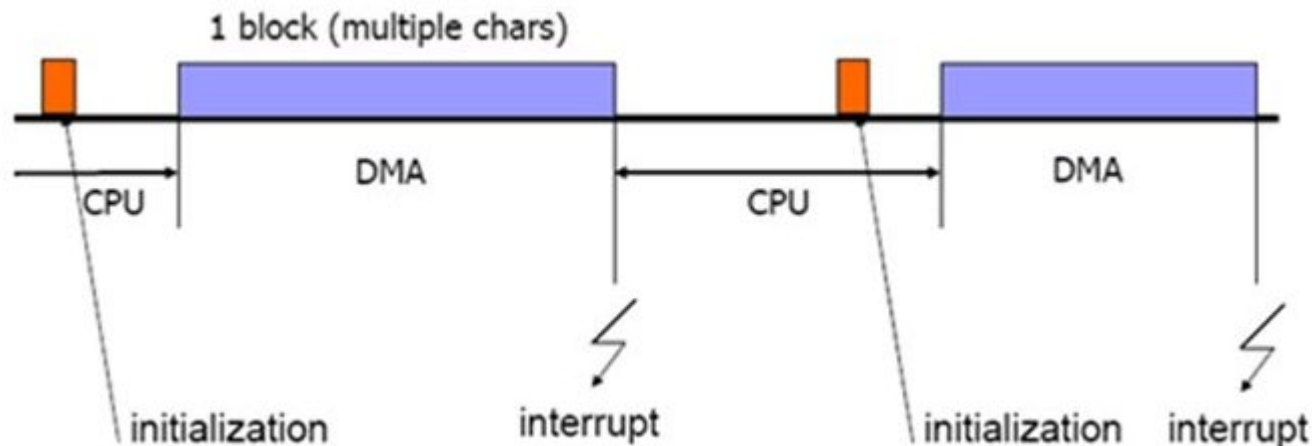


DMA I/O Access

- When a peripheral wishes to take part in an I/O transaction it asserts the **TransferRequest** input of the DMA controller (DMAC).
- In turn, the DMAC controller asserts the **DMArequest (BusReq)** to request control of the buses from the CPU.
- When the CPU returns **DMAgrant (BusAck)** to the DMAC, a DMA transfer takes place.
- Bus switch 1 is open and switches 2 and 3 are closed.
- The DMAC provides **TransferGrant** signal to the peripheral.
- The peripheral is able to communicate with the memory using the DMA
- When DMA operation is completed, DMAC hands control of the bus back to the CPU.

Burst Mode

- DMA controller seizes the system bus for the duration of the data transfer operation.
- Allows data to be moved at the fastest possible rate within the memory/bus/interface timing requirements
- CPU is effectively halted in the burst mode because it cannot use its buses.



Cycle Steal Mode

- DMA operations are interleaved with the processor's normal memory accesses.
- DMA can take place when CPU is busy.
- Transparent because transfer is invisible to the processor and no processing time is lost.

